

Bar-code Detecting and Decoding

Amid rapid technological advancement, barcode reading systems have become an essential component of product tracking, inventory management, and data processing. However, these systems face significant challenges when dealing with non-ideal images caused by varying lighting conditions, angular distortions, or visual noise, which often lead to failures in automated decoding.

This project aims to develop an intelligent barcode reading system using digital image processing techniques, enabling it to handle low-quality images and automatically enhance them before decoding. The project leverages open-source libraries such as "OpenCV" for image processing and "Pyzbar" for barcode decoding, while integrating adaptive algorithms to determine the need for specific preprocessing steps (e.g., rotation correction, contrast enhancement, or noise removal) based on the input image's characteristics.

The system incorporates decision-making capabilities to identify required processing steps, reducing computational time and improving accuracy and provides a simple user interface for image selection and visual result display, making it a practical tool for diverse applications.

Technical Steps for System Development:

1. Image Acquisition and Preprocessing:

- Import the origin barcode image from the user's device and upload it by OpenCV library.
- Convert RGB image to grayscale image for easier processing.
- Correct orientation:
 - Apply thresholding and find the largest contour representing the barcode or text.
 - Calculate the skew angle using `cv2.minAreaRect()`.
If the angle is different from 0° or 90°, the image is tilted:
 - If the angle is between -45 and -90 degrees, the image is rotated by $90 + \text{angle}$.
 - If the angle is between 0 and -45 degrees, the image is rotated by angle .

2. Image Enhancement:

Enhancing Contrast and Line Clarity.

- **Contrast Stretching:**
Increase contrast between black and white barcode lines by applying a linear transformation to stretch the pixel intensity range.
- **Histogram Equalization:**
Normalize brightness distribution to improve contrast by redistributing the histogram to expand dark and bright pixel values.
- **Power-Law Transformation:**
Correct uneven lighting (overly dark or bright images) using the equation $g(x, y) = c \cdot (f(x, y))^\gamma$ to adjust brightness.
- **High Boost Filter:**
Enhance sharp edges in barcode lines by applying the high boost filter and add it to the original image ($A = 1.7$).

3. Image Restoration:

Remove random and frequency noise.

- **Order-Statistics Filter (Median Filter):**
Remove "salt-and-pepper" noise while preserving line edges by replacing each pixel with the median value within a 3x3 window.
- **Gaussian Low-Pass Filter:**
 1. Convert the image to the frequency domain.
 2. Reduce random noise by smoothing the image using a Gaussian function and preserve edges by assigning higher weights to nearby pixels and lower weights to distant ones.
 3. Convert the image back to the spatial domain.
- **Notch Reject Filter:**
 1. Convert the image to the frequency domain (2D-DFT).
 2. Use Notch Reject Filter (NRF) to eliminate periodic noise from poor printing or scanning.
 3. Convert the image back to the spatial domain (IDFT).

4. Morphological Processing:

- **Closing (Dilation + Erosion):**
Fill small gaps in barcode lines by applying dilation followed by erosion using a small kernel (3×1 for vertical lines).
- **Opening (Erosion + Dilation):**
Remove small noise or distortions outside the barcode lines by applying erosion followed by dilation with an appropriate kernel (3×3 for small noise).
- **Thinning (Hit-or-Miss Transformation):**
Reduce line thickness to achieve uniform width by subtraction hit-or-miss transformation from the image.

5. Segmentation:

- Extract the Barcode from the Background.
- Use contour detection to locate the barcode area.
- Extract the largest rectangular bounding box containing the barcode.

6. Representation and Description:

- Convert the barcode to binary (black/white) format using Thresholding.
- Apply edge detection and line analysis to ensure the barcode is fully extracted:
 - **Sobel Gradient:**
Detect vertical and horizontal edges by applying horizontal and vertical Sobel filters, then combine the results.

7. Object Recognition:

- Use pyzbar / pylibdmtx libraries to decode the barcode.
- Identify the barcode type (e.g. QR code, UPC, EAN) and extract the encoded data.

Program Implementation and Results:

Firstly, the program provides a simple user interface for image selection. If the user doesn't select any thing, this message will be appeared: "No image has been selected.", else, origin image, processed image, barcode data and it's type will be appeared.

Test Case 01: (clear bar-code (binary))



Decoding Results:

Type: CODE128

Data: 220212022

Test Case 02: (clear bar-code (RGB))



Decoding Results:

Type: EAN13

Data: 0123456789128

Test Case 03: (dark bar-code)



Decoding Results:

Type: CODE128

Data: dynamsoft

Test Case 04: (bright bar-code)



Final Binary Image (Thresholding)



Decoding Results:

Type: EAN13

Data: 0123456789128

Test Case 05: (blurred bar-code)



Final Binary Image (Thresholding)

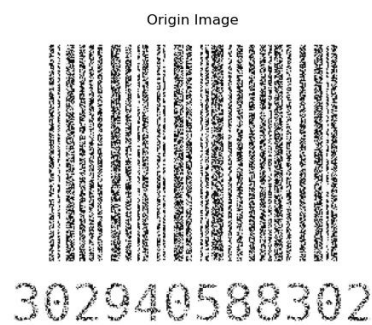


Decoding Results:

Type: EAN13

Data: 0076950450479

Test Case 06: (noisy bar-code)



Final Binary Image (Thresholding)

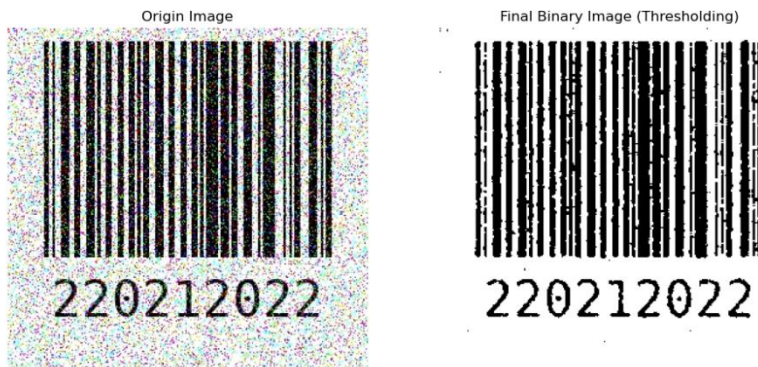


Decoding Results:

Type: CODE128

Data: 302940588302

Test Case 07: (salt and pepper noisy bar-code)



Decoding Results:
Type: CODE128
Data: 220212022

Test Case 08: (periodic noisy bar-code)



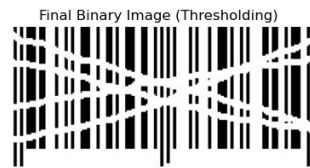
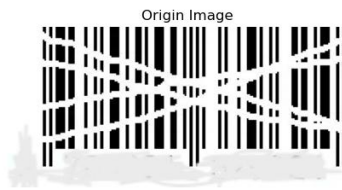
Decoding Results:
Type: CODE128
Data: 123456789

Test Case 09: (clear bar-code (text))



Decoding Results:
Type: CODE128
Data: C0016

Test Case 10: (noisy bar-code)

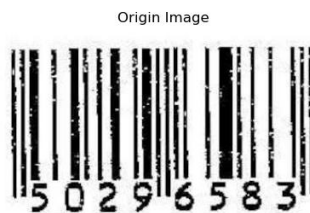


Decoding Results:

Type: EAN13

Data: 9780201379624

Test Case 11: (noisy bar-code)

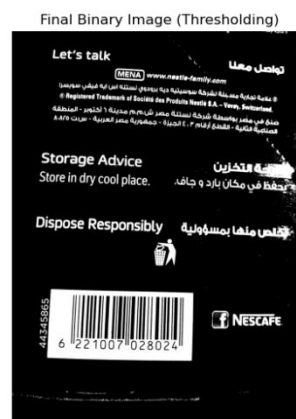
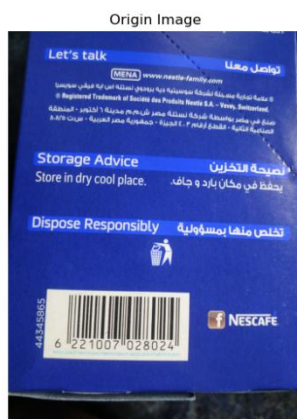


Decoding Results:

Type: EAN8

Data: 50296583

Test Case 12: (natural bar-code)

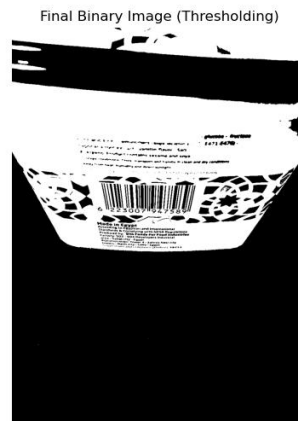


Decoding Results:

Type: EAN13

Data: 6221007028024

Test Case 13: (natural bar-code)

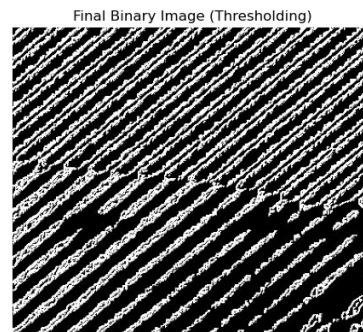
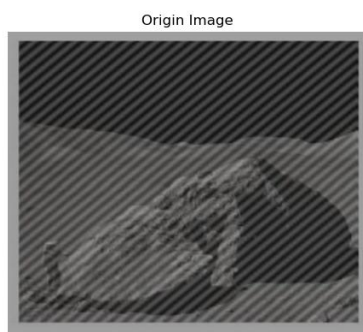


Decoding Results:

Type: EAN13

Data: 6223007947589

Test Case 14: (without bar-code)



No barcode detected

After importing the image and preprocessing, the program tries to read the barcode. If it successfully read without any additional steps, the program outputs the results and terminates. If reading fails, the program analyzes the image, identifies the necessary adjustments, and applies them.

This approach ensures maximum efficiency in barcode reading avoiding unnecessary operations and saving time by applying only the necessary adjustments when needed.

Written By: Heba Majed Kullab 220212022